

# Construct Target Array with Multiple Sum.

Ex1:

Target: 9 3 5

arr 1 1 1

total:  $\frac{3}{5} = \frac{9}{15}$

1 3 1

1 3 5

9 3 5

Same as target Arr

return ①

Ex2:

8 5 →

1 1

total =  $\frac{8}{15}$

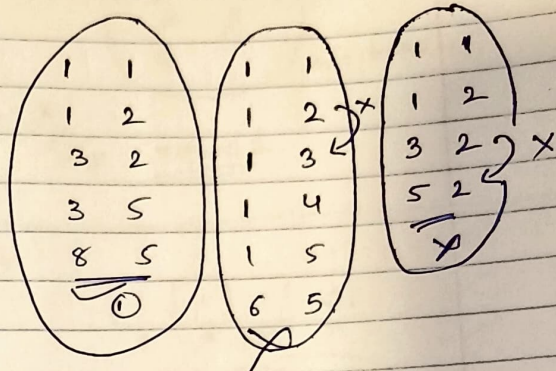
1 2

3 2

3 5

8 5

return 1



First thing: we should be able to first fulfill the smaller ele  
 $\therefore$  total = smaller element of target Arr.

Because if we try to fulfill the bigger element first then

9 3 5

1 1 1

9 + 1 + 1  
can never be 3 or 5.

9 1 1

Second thing:

we also need to ensure

1 2  $\times \rightarrow$  1 3  $\times$

$\hookrightarrow$  3 2  $\times \rightarrow$  5 2  $\times$

$\hookrightarrow$  3 5

$\begin{matrix} 1 & 2 \\ \vee & \times \\ 3 & 2 \end{matrix}$

if we see, we replace smallest element in existing Arr, ① is replaced with 3 (ideally).

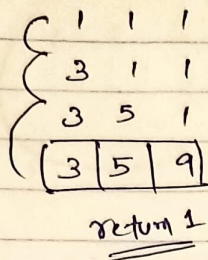
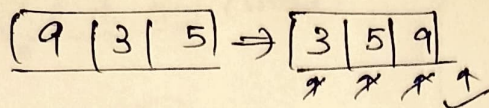
$\begin{matrix} 3 & 2 \\ \vee & \times \\ 3 & 5 \end{matrix}$

2 should be replaced with 5 & not 3 because 2 is smaller.

So we can sort the target Arr & then try to fulfill smaller elements.



Ex 1:



We can also use minheap. How?

Initially, only  $[1 | 1 | 1]$  in minheap

Pop minimum & insert total & update total

$total = 3$

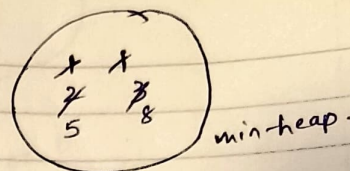
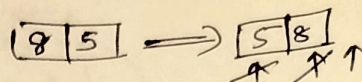
return 1.

on the target arr, check if each element is fulfilled if so increment i.

$total = 4$   
return 0 itself.

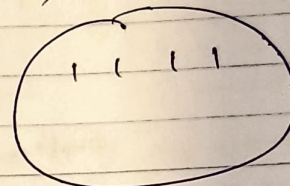
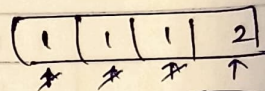
because even if we were able to achieve 4, Once 4 is achieved, there is no way, we can achieve 2.  
 $[4 + x \neq 2]$

Ex 2: let's see for this,



$total = 2$   
return 1

Ex 3:



total: 4

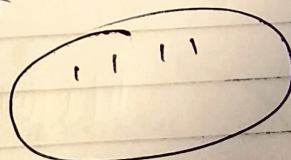
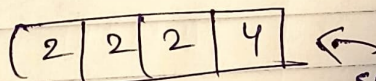
Cannot fulfill 1 or 2, so if it cannot fulfill smaller elements, even though if we fulfill bigger elements

Bigger etc  $+x \neq$  smaller

& if index ends or reaches size but unable to fulfill arr[i].

return 0

Ex 4:



sorted so even if single element is skipped or

target  $[i] < total$

return 0.

what logic can we derive from above examples?

1)  $total < target[i]$  operations (on minheap) & update total.

2)  $total = target[i]$  operations + (itt)

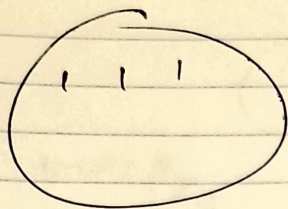
3)  $total > target[i]$  return 0

let's see edge cases on (3).



Ex 1

1 | 1 | 5 (sorted).



total: 3

total > target[i]

return 0.

should we?

As we know

1 1 1  
1 1 3

1 | 1 | 5

should ideally return 1.

then?

Skip 1 & check for all positive elements > 1

let's see

Ex 2:

1 | 1 | 1 | 1 | 2



total = 4

if i reaches end return 1 or 0?

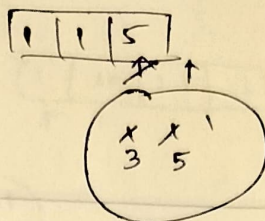
Last element fulfilled, return 1 else return 0.

also

target[i] != 1 && total > target[i] return 0.

Right?

let's take one more ex



total: 5

return 1 or 0?

ideally elements should be

But from minheap Arrangements

1 1 1  
1 1 3  
1 1 5

1 1 1  
3 1 1  
3 5 1

why! we previously saw how

(1, 2)

(3, 2) (1, 3)

ideally this is correct But we are not able to achieve ans. using this way.

even so if we change a logic,

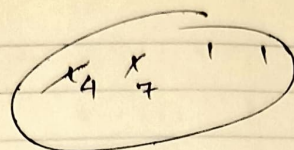
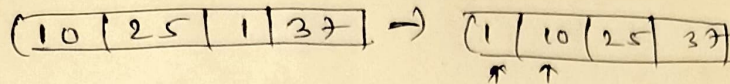
① total < target[i] operations (minheap & update total).

② total == target[i] operations + (i++).

③ target[i] != 1 && target[i] < total return 0 else if target[i] == 1 i++.



Ex:

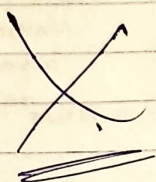


total =  $\frac{7}{13}$

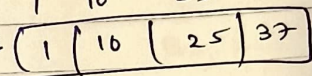
Not able to fulfil 10  
so return 0 ?

But if we see

|   |    |    |   |
|---|----|----|---|
| 1 | 1  | 1  | 1 |
| 1 | 4  | 1  | 1 |
| 1 | 7  | 1  | 1 |
| 1 | 10 | 1  | 1 |
| 1 | 10 | 13 | 1 |
| 1 | 10 | 25 | 1 |



ideally it should  $\rightarrow$  return 1



So 100% our logic is getting failed.

we also understood going from 1 1 1 to target Arr

Seems impossible using Min-heap Approach because either we had

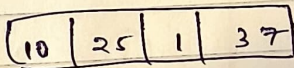
to go brute-force Approach, (Minheap App is not working)

& brute-force says: For each ~~element~~ total,

we have to iterate & check & keep track of mark-down Elements. & that again

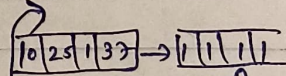
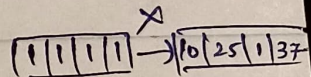
is way too complicated until we choose to break-down into sub-problems efficiently handling proper edge case scenarios.

So our previous Approach failed for this scenario:



So then how to address?

|    |    |   |    |
|----|----|---|----|
| 1  | 1  | 1 | 1  |
| 4  | 1  | 1 | 1  |
| 7  | 1  | 1 | 1  |
| 10 | 1  | 1 | 1  |
| 10 | 13 | 1 | 1  |
| 10 | 25 | 1 | 1  |
| 10 | 25 | 1 | 37 |



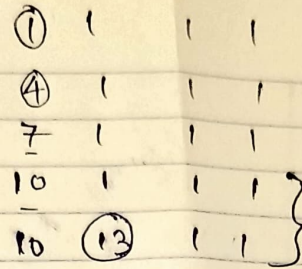
let's try

But is there other way to address this problem? Can we think of going other-way around?



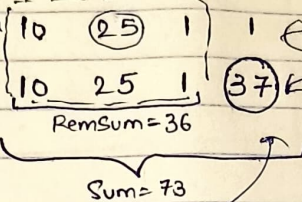
so on & so fourth until it becomes 1

Similarly for this level too



remain unchanged

All the elements will remain unchanged except max element.



only MaxEte gets changed

Operations being performed.

$$\begin{aligned} \text{MaxEte} &= 37 \\ \text{Sum} &= 73 \\ \text{RemSum} &= \text{Sum} - \text{MaxEte} \\ &= 36 \end{aligned}$$

$$\begin{aligned} \text{Element} &= \text{MaxEte} - \text{RemSum} \\ &= 37 - 36 \\ &= 1 \end{aligned}$$

\* pick MaxEte, calculate Remaining Sum, & calculate newEte value

$$\begin{aligned} \text{MaxEte} &= 13 \\ \text{Sum} &= 25 \\ \text{RemSum} &= 12 \end{aligned}$$

$$\begin{aligned} \text{Element} &= \text{MaxEte} - \text{RemSum} \\ &= 13 - 12 \\ &= 1 \end{aligned}$$

How? let's see.

Now Edge cases:

if ( $\text{RemSum} \leq 0$  ||  $\text{RemSum} \geq \text{MaxEte}$ )  
return 0;

Ex: 1) 1 1 1 2

$$\begin{aligned} \text{Sum} &= 5 \\ \text{MaxEte} &= 2 \\ \text{RemSum} &= 3 \\ \text{Element} &= -1 \end{aligned}$$

We want this condn to satisfy:

$$\text{MaxEte} = \text{Ete} + \text{RemSum}$$

but if

$$\text{RemSum} \geq \text{MaxEte}$$

$$\text{MaxEte} \neq \text{Ete} + \text{MaxEte}$$

Element < 1

& so here it should not be

Ex2: 2 2 2

$$\begin{aligned} \text{MaxEte} &= 2 \\ \text{RemSum} &= 6 \end{aligned}$$

$$\begin{aligned} \text{Element} &= 2 - 6 \\ &= -4 \\ &\leq 0 \\ &\approx -1 \end{aligned}$$

because  $\text{RemSum} > \text{MaxEte}$

Ex3: Secondly ( $\text{RemSum} \leq 0$ )

$$\begin{aligned} 0 \quad 0 \quad 4 \\ \text{Sum} &= 4 \\ \text{Max} &= 4 \\ \text{RemSum} &= 0 \end{aligned}$$

only when  $\text{sum} = \text{MaxEte}$

$$\text{MaxEte} < 0$$

$$\begin{aligned} \text{Ete} &= \text{MaxEte} - \text{RemSum} \\ &= 4 - 0 = 4 \\ \text{Element} &\neq \text{MaxEte} \end{aligned}$$



Now for doing this, will make use of. Max-heap

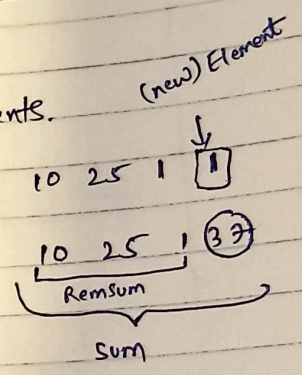
|    |    |   |    |
|----|----|---|----|
| 1  | 1  | 1 | 1  |
| 4  | 1  | 1 | 1  |
| 7  | 1  | 1 | 1  |
| 10 | 1  | 1 | 1  |
| 10 | 13 | 1 | 1  |
| 10 | 25 | 1 | 1  |
| 10 | 25 | 1 | 37 |

Everytime, we are picking MaxEle, computing new value of Ele & pushing it back so (popping & pushing) Altogether + Maintaining the order  
 → Heap is go-to choice. else in Arr, shifting might take overhead. T.C

Steps:

- 1) Create Max-heap
- 2) Insert all target Arr elements & calculate sum of all Elements.

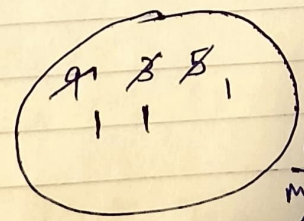
3) loop until  $(p.top() \neq 1) \{$   
 pick MaxEle  $(p.top()) + p.pop()$   
 compute Remaining Sum  $(RemSum = Sum - MaxEle)$   
 calculate newEle value  
 & insert the newEle back into heap  
 + compute new Sum = RemSum + Element  
 }  
 Either line, ensure handling of edge cases.  
 edge cases: return 0



return 1.

Ex: DRY RUN

9 3 5



Sum = 17  
 9 5

MaxEle = 9  
 RemSum = 8  
 Ele = 1  
 MaxEle = 5  
 RemSum = 4  
 Ele = 1  
 MaxEle = 3  
 RemSum = 2  
 Ele = 1

until  $(p.top() \neq 1) \{$   
 MaxEle =  $p.top()$   
 $p.pop()$   
 RemSum =  $Sum - MaxEle$   
 =  
 Element =  $MaxEle - RemSum$   
 if (Element >= MaxEle || Element <= 1)  
 return 0.  
 Sum =  $RemSum + Element$   
 $p.push(Element)$ ;  
 }

MaxEle  $p.top() = 1$  stop return 1.



CODE :

```
bool isPossible (vector<int> & target) {
```

```
    // Max-heap
    priority_queue<int> p;
    int sum = 0;
```

```
    for (int i = 0; i < target.size(); i++)
    {
        p.push (target[i]);
        sum += target[i];
    }
```

```
    int MaxEle, RemSum, Element;
```

```
    while (p.top() != 1) {
        MaxEle = p.top();
        p.pop();
        RemSum = sum - MaxEle;
        // Element = MaxEle - RemSum.
        // & we know Element < 1 x
        // Element > MaxEle x
        // it is only possible
        //
        // RemSum <= 0
        // or
        // RemSum >= MaxEle
```

```
    if (RemSum <= 0 || RemSum >= MaxEle)
        return 0;
```

```
    Element = MaxEle - RemSum;
    sum = RemSum + Element;
```

```
    p.push (Element);
```

```
    } // while loop ends
```

```
    return 1;
```

```
    } // func ends.
```

But Tle  
Why?

TLE :

Why so?

Ex: [1, 20]  
[1, 19]  
[1, 18]  
[1, 17]  
[1, 16]

[1, 1] stops.

What if

[1, INT-MAX]  
[1, INT-MAX-1]  
}  
}  
[1, 1]

Ex2:

[10 | 25 | 1 | 37]

what is happening here

|    |    |   |    |
|----|----|---|----|
| 1  | 1  | 1 | 1  |
| 4  | 1  | 1 | 1  |
| 7  | 1  | 1 | 1  |
| 10 | 1  | 1 | 1  |
| 10 | 13 | 1 | 1  |
| 10 | 25 | 1 | 1  |
| 10 | 25 | 1 | 37 |

Too much  
cumbersome.

Approach

Any faster Approach:

[10 % 3 = 1]

Element = MaxEle % RemSum





Another Ex:

20, 5

$$\left\{ \begin{array}{cc} 0 & 5 \\ 5 & 5 \\ 10 & 5 \\ 15 & 5 \\ \underline{20} & 5 \end{array} \right\} \text{ stops.}$$

Similarly:

21, 5

$$\left\{ \begin{array}{cc} \underline{1} & \underline{5} \\ 6 & 5 \\ 11 & 5 \\ 16 & 5 \\ 21 & 5 \end{array} \right\} \text{ stops}$$

$$\left\{ \begin{array}{cc} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ \underline{1} & \underline{5} \end{array} \right\} \text{ stops}$$

10 steps

Ex1:

[1, 20]

⋮

MaxEle=20

$$\text{Ele} = 20 \% 1$$

$$= 0.$$

[0, 0]

$$\left\{ \begin{array}{l} \text{Ele} = 0 \\ \text{RemSum} = 1 \\ \text{return } 1 \end{array} \right.$$

21, 5

Better way

$$1 \Rightarrow \text{we can stop here}$$

$$\text{Ele} = \text{RemSum}$$

$$\text{MaxEle} \% \text{RemSum}$$

[1, 0] or

$21 \% 5 = 1$

$$\left\{ \begin{array}{l} \text{Ele} = 0 \\ \text{RemSum} = 1 \\ \text{return } 1 \end{array} \right.$$

$$\text{Ele} = 5 \% 1$$

$$= 0$$

Ex2:

1 2 6

1 2 0

$$\text{Ele} = 6 \% 3$$

$$= 0$$

RemSum=3

$$\left\{ \begin{array}{l} \text{Element} = 0 \\ \text{RemSum} \neq 1 \end{array} \right.$$

$$\text{return } 0;$$

Ex3: 1 1 4

1 1 0

$$\text{Ele} = 4 \% 2$$

$$= 0$$

RemSum=2

$$\left\{ \begin{array}{l} \text{Ele} = 0 \\ \text{RemSum} \neq 1 \end{array} \right.$$

$$\text{return } 0;$$

CODE: // Only changes.

// Edge case

Element = MaxEle % RemSum

// Another edge case

if (Element == 0) {

if (RemSum != 1)

return 0;

else return 1;

}

Still integer can overflow  
so take long long everywhere,

[1, INT MAX, INT MAX]

sum can be overflow.

etc.